

G-one

How the system is built: the wearable, the phone, the link between them and three generations of hardware.



A wearable senses. The phone decides and explains. Nothing is guessed, and nothing is hidden.

This document describes what G-one actually does today, part by part. Where something is a design asset or a presentation claim rather than working behaviour, it says so in the same breath.

The rule everything follows.

Deterministic rules decide health events; the language model only rewords them. If the model never loads, every alert still arrives with usable wording.

Inside this document

Pages	Contents
02 – 04	System architecture, the wearable and its firmware, the Bluetooth link
05 – 07	The Android application, alerts and SOS, the on-device language model
08 – 11	The three hardware prototypes, and every 3D file listed
12 – 14	Tools, privacy, what is proven, what is not, and the next steps

Written from the code, the recorded bench and phone evidence in VERIFICATION.md, and the README. The full text version is docs/G-ONE-COMPLETE-DESCRIPTION.md.

The three parts

One device measures, one device thinks, and a few optional services reach outside the phone.

Part	What it is responsible for
1 · Wearable	An ESP32-S3 with four sensors. It measures, builds one line of text twice a second, and sends it. It stores nothing and decides nothing.
2 · Android phone	Everything else: parsing, validation, storage, the 13 detection rules, alerts, sessions, reports, the local language model and every screen.
3 · Optional services	Only if switched on: city air quality, a consented emergency snapshot behind an NFC or QR link, and SMS or calls to one saved contact.

A reading becomes an alert

1 · Store The reading is written to the database first	2 · Detect Rules run over the recent window	3 · Record The event is stored with its explanation	4 · Alert The wearer is told immediately	5 · Reword The model may improve it, later
--	---	---	--	--

The order is the design. Storing comes before detecting, detecting before alerting, and the model last and off to one side, where a slow or broken model cannot delay or weaken an alert.

Who owns what, in the code

Responsibility	Where it lives
Reading sensors and sending lines	firmware/g_one_wearable
Framing bytes, checking units and ranges	health/source/SensorPacketParser.kt
Connecting, reconnecting, clock sync	health/source/ble/
Storing, aggregating and calling the detector in order	health/service/MonitoringPipeline.kt
Deciding whether something is wrong	health/detect/ and health/domain/AnomalyThresholds.kt
Wording an alert, then improving it in the background	health/explain/ and the explanation worker
Sharing one language model between chat and tools	ai/repository/

Responsibility	Where it lives
Anything that leaves the phone	health/sos/, health/emergency/, health/environment/

What is on the strap

Four sensors, two I²C buses and a firmware built from the owner's own working bench sketch.

Part	Measures	Connection
ESP32-S3-N16R8	—	Runs the firmware and Bluetooth LE
MAX30100	Pulse, blood oxygen	I ² C bus 0 · SDA 8, SCL 9 · 0x57
MPU6050 (GY-521)	Movement and impacts	I ² C bus 1 · SDA 6, SCL 7 · 0x68
DS18B20	Skin temperature	1-Wire GPIO 4, 4.7 kΩ to 3V3
EMG module	Muscle activity	Analog GPIO 1, must stay within 0–3.3 V

Two separate I²C buses are deliberate: the pulse sensor is read every millisecond on its own task, so giving the motion sensor its own bus means neither waits for the other.

Three corrections the app needed

Found in the bench sketch	Why it mattered	Fix
Heart rate sent as a decimal	The app reads whole numbers, so every reading would have been dropped	Rounded
Accelerometer at ±2 g	The fall rule needs a harder impact than ±2 g can report at all	±8 g
A dead sensor kept its last value	A stopped sensor would be shown as a live reading	Dropped after 5 s

A stuck heart rate, and the guard against it

The MAX30100 library clears its rate from only one internal state. On the bench it stuck after the board was knocked and reported **0.3 BPM with nothing on the sensor**; after a real pulse it would repeat that pulse indefinitely. The firmware now sends a rate only when the library's own beats back it up: five beats found, the newest within 2.5 seconds, and the rate within 20 % of what those beats imply.

Live only. There is no SD card on this build. Nothing is kept on the board, so a reading made while the phone is out of range is lost and appears as a gap. The app still speaks the fuller store-and-forward protocol, so a wearable that keeps readings would work unchanged.

Power settings, and the fix still outstanding

Measuring its own I²C lines at start-up showed the MAX30100 module pulls that bus to its internal **1.8 V**, while the ESP32-S3 needs about 2.5 V to read a “1”. That is why the pulse sensor answered on some boots and not others, and why no heart rate has ever reached the app.

Setting	Value, and why
Outstanding hardware fix	2.2 kΩ from SDA to 3V3 and from SCL to 3V3. Until fitted, the firmware prints TOO LOW at start-up
CPU speed	80 MHz instead of 240 — less current and heat; sensors and Bluetooth are unaffected
Bluetooth power	3 dBm instead of 9 — smaller current spikes
Pulse sensor bus	100 kHz instead of 400 kHz — four times as long for a weak line to rise
A sensor that does not answer	Asked three times at start-up, then every 5 s, without a restart

One line, twice a second

Bluetooth LE using the HM-10 serial profile: service FFE0, characteristic FFE1.

```
HR:72,SPO2:97,STEMP:33.90,MOT:1.02,EMG:1234,
EMGPK:1810,EMGBITS:12,ST:5D,TS:1726470000000
```

Key	Meaning	Accepted
HR	Heart rate, whole beats per minute	20 – 300
SPO2	Blood oxygen, %	50 – 100
STEMP	Skin temperature, °C	20 – 42
TEMP	Core temperature — not sent by this hardware	25 – 45
MOT	Hardest movement since the last line, g	0 – 16
EMG / EMGPK	Muscle level and peak, raw ADC counts	0 – 4095
EMGBITS	Resolution of those counts	must be 12
ST	Status bits, hex	see below
TS	When the reading was taken	2 min ahead / 30 days back

Status bits, and what the app says about them

Bit	Meaning	When clear, the app says
0x01	Pulse sensor found	“Heart-rate and oxygen sensor not detected”
0x02	A pulse is being found	“Heart-rate sensor is not touching skin”
0x04	Thermometer found	“Skin temperature sensor not detected”
0x08	Motion sensor found	“Motion sensor not detected — falls cannot be detected”
0x10	EMG signal looks connected	“Muscle sensor pads look detached”
0x20	Card working	Always clear on this build
0x40	Clock set by the phone	“Wearable clock not set yet”

Units are checked, not assumed. A skin temperature sent in Fahrenheit lands outside 20–42 and is dropped rather than read as a fever. EMG counts are refused unless the line says they are 12-bit. A missing field means “not measured” and never becomes a zero.

Phone to wearable: **T:<epoch ms>** sets the clock on every connection. ACK and STOP belong to the store-and-forward protocol and are ignored by this firmware. Framing is newline-terminated ASCII, and notifications split or merged by Bluetooth are reassembled before parsing.

Where everything is decided

Kotlin and Jetpack Compose, package com.gone.ai, with the detection engine kept free of Android.

Package	What it holds
health/source/ · ble/	Parsing, validation, 5-second aggregation, the Bluetooth link and its watchdog
health/service/	Foreground monitoring service, the ordered pipeline, preferences, reminders
health/detect/ · domain/	The 13 rules, thresholds, baselines and risk scores — plain Kotlin, no Android
health/explain/	Written explanations, tiers, and validation of any model rewrite
health/session/	Sessions, report building, PDF layout and writing
health/sos/	Automatic emergency texting and calling
health/emergency/ · nfc/	Consented snapshot, NFC tag writing, QR codes
health/environment/	Optional city air quality
ai/ · chat/	Shared model runtime, prompts, context budgets
ocr/ · pdf/ · circle/	Text from images and documents, screen capture tools
data/library/	Room database and the Memory Vault
voice/	Speech in and out, with TalkBack labels throughout the app

The 13 rules

Rule	Watches for
spo2.critical · spo2.sustained	Critically low oxygen, and a sustained low pattern
hr.high · hr.low	Sustained fast or slow heart rate
temp.fever	Raised core temperature, when a core sensor exists
env.heatStress · env.dehydrationRisk	Combined heat and dehydration indicators
resp.distress · cardio.strain	Combined respiratory and cardiovascular patterns
motion.fall	A hard impact, with supporting conditions

Rule	Watches for
wellness.fatigue · emg.sustainedHigh	A fatigue pattern; muscles held tense
baseline.deviation	A departure from this person's own baseline

Skin temperature never feeds a fever rule. Risk scores organise findings; they are not probabilities of illness. The fall threshold is **5.0 g**, raised from 2.5 g so ordinary knocks do not occupy the cooldown before a harder impact — an engineering setting, not a validated fall classifier.

Telling someone, and writing it down

An alert always arrives with usable wording. An SOS only ever follows a confirmed live reading.

Automatic SOS

Question	Answer
What sends one?	Every confirmed live anomaly from the wearable, including low and moderate ones. Motion additionally needs a finite impact of at least 5.0 g
What never sends one?	Simulated readings, replayed history, air quality
Is there a countdown?	No. Alerts raised within 1.5 s are gathered into one message
Does it call as well?	Optional, separately permissioned; the text goes first
What does it need?	Opt-in, a saved contact, SMS permission, a working SIM and signal
What does it prove?	Nothing about receipt: a successful send is not proof the contact read it

Sessions and the PDF report

A session records a stretch of monitoring. Ending it builds a report from what was actually stored: duration, reading count, the range of each channel, five representative points, charts, and the alerts raised. It states plainly when a channel received nothing, and counts gaps longer than a minute.

Checked end to end on the hardware. A 19-minute session on the real wearable produced a two-page PDF whose every figure matched the stored readings: skin temperature 29.8 – 32.5 °C, hardest movement 9.5 g, EMG average 422 and peak 4095, one 85-second gap, one alert.

The last section of the report is the local model's summary of the deterministic observations above it, labelled as such. One limit found in testing: the validation forbids new numbers, advice and diagnoses, but does not catch a statement that is simply wrong in words.

A language model that cannot cause harm

It runs on the phone's CPU, it is shared between every tool, and it is never allowed to decide anything.

Item	Value
Model	Qwen2.5-1.5B-Instruct, GGUF Q4_K_M, 1,117,320,736 bytes
Runtime	Vendored llama.cpp / ggml through a JNI bridge, CPU only, 4 threads
Context	4,096 tokens — 3,568 for the prompt, 512 reserved for the answer
Sharing	One shared instance; chat and tools take leases and requests are serialised
Release	Freed 60 seconds after the last lease closes
Delivery	Bundled in the APK, copied to app-private storage on first use
Recorded speed	28.9 tokens/s prompt processing, once, on a Samsung SM-S721B

Bundling the model is why the release APK is about **1.13 GiB**, of which the model is roughly 94 %. It buys a first run with no download, on a phone with no connection.

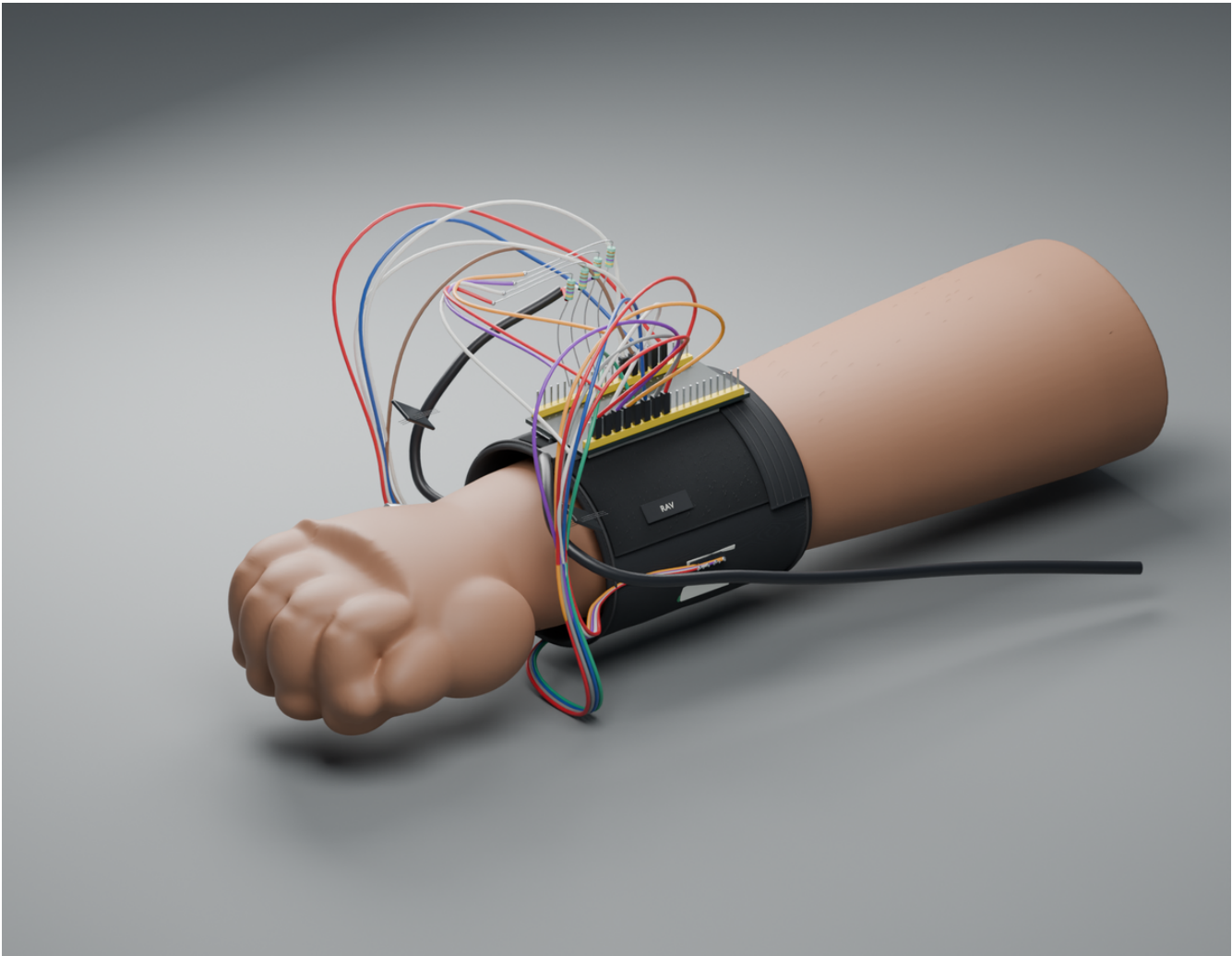
What the model is allowed to do

- **Health explanations:** reword an explanation that already exists. The result is validated — no new numbers, no advice, no diagnoses — and rejected text leaves the original in place.
- **Documents and tools:** summarise, explain or build a quiz from text the wearer chose.
- **Assist:** general chat, which is **not** covered by that strict validation and can be wrong.

It never decides. The model is not asked whether a reading is dangerous, never sets severity and never chooses what the wearer should do. Every one of those decisions belongs to a named rule with a reviewable threshold.

Phase 1 — the strap that exists

The prototype the firmware actually runs on, reconstructed in Blender from eleven photographs.



Blender reconstruction of the worn Phase-1 strap: inverted ESP32-S3 board, exposed headers, prototype wiring, four axial resistors and the temperature probe.

Item	Detail
Scenes	01 Worn Phase-1 prototype · 02 Opened strap, sensor inspection
Modules modelled	ESP32-S3 board, Muscle BioAmp Patchy, MAX30100/30102 breakout, MPU6050 GY-521, temperature probe
Size	1,684 objects, 11 packed reference photographs
Renders	phase1-worn, phase1-hardware, phase1-inside, phase1-opened
Scale cue	Nominal 2.54 mm header pitch

Limits: photo-derived, not a scan. Strap dimensions, hidden mounting, resistor values and obscured routing are approximations, and the closed fist is modelled approximately because the photographs do not show it. The digital wiring is not a verified electrical netlist.

Later phase — the custom PCB

What the strap could become as one integrated board, rather than development boards stacked together.



Rev B assembled: pouch cell on its removable carrier above the electronics, away from the skin-facing sensors.

Item	Detail
Board	44 × 64 × 1 mm, six mounting holes; one Blender unit = 1 mm
Contents	905 objects in 12 collections: laminate and plating, ESP32-S3-WROOM-1-N16R8, MPU6050, microSD socket, EMG front end, charger and buck-boost, USB-C and switches, skin-side MAX30102, DS18B20 thermal island, routing relief, optional battery, studio
Rev B added	Pouch-cell assembly and removable carrier, three editable harness curves, TP4056 charger with separate DW01A / FS8205A protection, exposed FR-4 edges and copper lands, a corrected capacitor overlap

Item	Detail
Renders	Eight 1800 × 1800 Cycles views, from assembled down to sensor macro
Rev A kept	G-one-PCB-before-enhancement.blend, unchanged

This model must not be used to order PCB fabrication. Visible copper is surface detail, not routed nets. The cell envelope is provisional, connector polarity and the thermistor circuit need checking, and the EMG analog design is unresolved. It also shows a MAX30102 while the firmware drives a MAX30100 — that substitution needs a different driver.

Final phase — the Jeevan Core patch

A reusable body patch built as eleven mechanical layers that pull apart, with a working web viewer.



The eleven layers exploded: enclosure and light guide at the top, electronics and battery in the middle, sensor flex, electrodes, adhesive and peel liner at the skin side.

#	Layer	What it is
0 1	Protective enclosure	Pearl polymer shell, identity panel, USB-C opening
0 2	Status light guide	Carries the PCB LED to the front indicator
0 3	Perimeter gasket	Elastomer seal — sealing performance untested

#	Layer	What it is
04	Main electronics	ESP32-S3 N16R8, ECG and EMG front ends, MPU6050, BME280, charging, microSD
05	Rechargeable cell	Protected pouch envelope; capacity awaits cell selection
06	Inner support frame	Battery pocket, board supports, screw bosses, flex passage
07	Flexible sensor interface	Polyimide substrate, optical and temperature sensing
08	Skin-side spacer	Silicone carrier with optical and electrode apertures
09	Electrodes and optics	Two ECG, two EMG, one BioZ and one temperature contact
10	Replaceable adhesive	Die-cut layer; material qualification pending
11	Peel-away liner	Protects the adhesive before use

Provisional $30 \times 50 \times \sim 13$ mm body over a $24 \times 40 \times 1$ mm board. Frame 1 is assembled, frame 121 exploded, exported as an **Explode** animation clip. The React and three.js viewer in web/ offers drag, a slider, a layer index, keyboard and screen-reader support, and a full text fallback if WebGL fails.

Limits: the illustration's 8 mm thickness and 28×24 mm PCB are not asserted. Electrode spacing follows the concept image and is not established as workable for ECG, EMG or BioZ. Battery life, ingress protection, biocompatibility and adhesive lifetime are not established, and the web copy deliberately avoids the unvalidated "5–7 day" and "IP67" claims.

Every file, listed

Three models, each with an editable Blender source, a self-contained GLB, renders, build scripts and validation reports.

Phase 1 strap

File	Size	Detail
G-one-Phase-1-Strap.blend	6.7 MB	Editable source, both scenes, 11 packed photos
G-one-Phase-1-Strap.glb	36.3 MB	1,089 meshes · 1,483,516 triangles · 28 materials
G-one-Phase-1-Strap-web.glb	20.6 MB	9 groups · 694,586 triangles, simplified for the web
4 final renders + 2 previews	~1–3 MB each	worn, hardware, inside, opened
7 scripts	1–23 KB	build → refine → finish → final check → render, plus two GLB exporters
4 JSON reports	< 1 KB	model, geometry audit, both GLB exports

Custom PCB

File	Size	Detail
G-one-PCB.blend	2.9 MB	Rev B, 905 objects in 12 collections
G-one-PCB.glb	15.5 MB	1,157 meshes · 324,162 triangles · 24 materials
G-one-PCB-before-enhancement.blend	2.8 MB	Preserved Rev A
G-one-PCB-before-enhancement.glb	11.1 MB	892 meshes · 233,574 triangles · 15 materials
8 renders + 1 reference	~2.3–3.9 MB each	assembled, isometric, top, bottom, side, sensors, esp32-power, detail
8 scripts	1–22 KB	Rev A builder, Rev B enhancement chain, cameras, exporter, audits
5 JSON reports	< 1 KB	inventory, validation, enhancement, mechanical audit, GLB export

Jeevan Core

File	Size	Detail
Jeevan-Core.blend	483 KB	Editable source, three cameras, layer animation
Jeevan-Core.glb	18.6 MB	11 named layer parents · 333,846 triangles · Explode clip
3 renders	~2.2 MB each	assembled, exploded, skin side
layers.json	2 KB	Stable IDs, titles, descriptions, explode distances
4 scripts	1–17 KB	build, refine, export, render
2 JSON reports	< 8 KB	export report and glTF validation
web/	—	React viewer: CoreExplorer.jsx, layers.json, style.css

Common to every GLB: metres, glTF Y-up, self-contained buffers, no Draco or Meshopt decoder needed, portable colour/metallic/roughness materials. Each export was re-imported to confirm mesh counts and bounds, and each source .blend's SHA-256 was checked unchanged afterwards.

Rebuilding: the construction scripts start a new scene. Never run them over a hand-edited final file — open the saved .blend, or follow the chain documented in that model's README.

Testing without hardware, and what leaves the phone

Two tools stand in for the missing half of the link; every outbound path is listed.

Tool	What it does
tools/wearable-emulator/emulate.py	Plays the wearable from a PC. --selftest runs the whole sync protocol against a simulated phone on a virtual clock: 10 scenarios, including going out of range, the app being killed mid-replay, a restart before the clock is set, and a flaky link. No hardware needed
tools/wearable-check/check_wearable.py	The opposite: the PC acts as the phone against the real board, checking every field against the app's own rules and comparing each line received over Bluetooth with the line the board printed on USB

The emulator's self-test found a real protocol bug during development: a status line sent before the records it described meant catch-up could never end.

What can leave the phone

Path	Local by default	What can leave
Detection and the language model	Yes	Nothing
OCR, Vault, sessions and reports	Yes	Only what the wearer exports
Sleep and stress journal	Yes	Nothing
Air quality, if switched on	No	City or centre coordinates; the request reveals an IP
Emergency snapshot, if switched on	No	The fields the wearer selected
Automatic SOS, if switched on	Carrier	SMS and optional call to the saved contact
Voice recognition	Depends	May use online recognition if no offline pack exists

Say it accurately. The database is app-private SQLite and is **not encrypted**. Cloud backup is switched off, so readings, chats and reports are not copied to a cloud account. It is wrong to say nothing ever leaves the phone in every configuration.

What is proven, and what is not

Software checks, bench observations and clinical validation are three different things.

Measured or observed

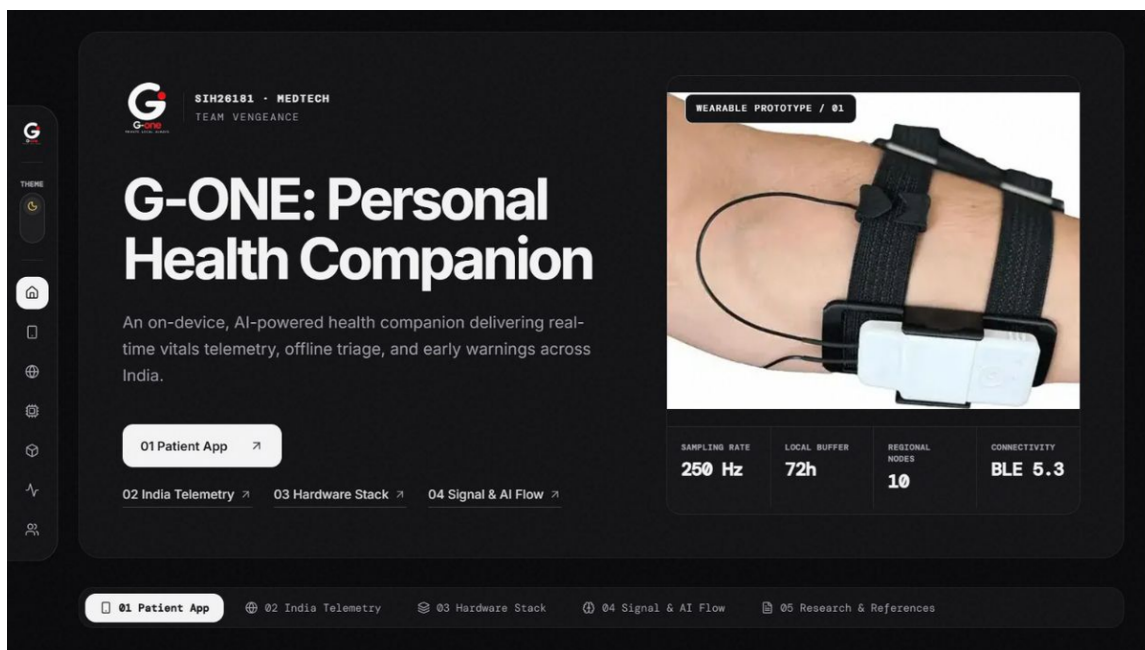
Evidence	Result
JVM unit tests	590 tests in 66 classes, 0 failures
Android lint	0 errors
Instrumented tests	19 of 19 on a Samsung SM-S721B
Firmware build	Compiles with no warnings from the sketch; about 21 % of the 3 MB app partition
Protocol self-test	10 of 10 scenarios
On the phone and board	Connects and sets the clock within seconds; 112 stored values checked against the board's own log with none disagreeing; Stop reaches the board in about 2.5 s; a 9.47 g jolt raised a fall alert; a 19-minute session produced a PDF matching the stored readings

Not established

- **A heart rate from the wearable has never reached the app.** The pulse sensor's bus sits at 1.8 V and the 2.2 kΩ pull-ups are not yet fitted.
- **Board power is unstable:** repeated brown-outs, a hot board and sensors that vary from boot to boot point to a wiring fault rather than firmware.
- A real SOS text and call have not been sent from the phone.
- Fall sensitivity, false-positive rates, battery life, sustained generation speed, memory use and thermal behaviour are unmeasured.
- The PCB and Jeevan Core designs have had no electrical review or fabrication.

Website claims, and the road ahead

The public site presents the intended product; this document describes the built one.



The public site at g-one.vercel.app, captured 22 September 2026.

The website says	This build actually does
250 Hz sampling	Two readings a second over Bluetooth; one stored reading every five seconds
72-hour offline buffer	Live only — nothing is stored on the wearable; out-of-range readings are lost
Encrypted local SQLite	App-private SQLite, not encrypted
HRV, arrhythmia triggers, siren, multi-contact SOS	Heart rate, SpO ₂ , skin temperature, motion and EMG; 13 rules; SMS and an optional call to one saved contact

Next meaningful steps

Limitation	Next step
Pulse sensor never delivers a rate	Fit the 2.2 kΩ pull-ups, then confirm a rate against a manual count
Unstable board power	Find the heat source, then decoupling and firm power rails
Live-only transport	Implement and verify buffering and gap recovery on the board
Engineering thresholds	Evaluate sensitivity and false positives with proper oversight

Limitation	Next step
1.13 GiB APK	Pin the model's provenance; weigh smaller models against quality
Unencrypted database	Evaluate encryption, key lifecycle and migration
Concept hardware	Electrical review, tolerances, fabrication and assembly testing

No claim is made for certification, guaranteed fall detection, diagnostic accuracy, waterproofing, multi-day battery life or universal device compatibility. G-one records sensor readings and explains them; it does not diagnose.

Source basis: the repository code as committed, docs/G-ONE-COMPLETE-DESCRIPTION.md, VERIFICATION.md, the three hardware READMEs and their export reports, and the public website as captured on 22 September 2026.